



Capítulo 12 - Integração entre sensores, desvios e tração

Receber uma informação, verificar a etapa da operação e emitir somente os comandos permitidos.

Introdução	Nos capítulos anteriores, estudamos os elementos separadamente. Agora vamos reunir seleção de percurso, comando de desvio, confirmação, eventos de sensores e solicitações de tração em uma sequência didática.
Objetivo	Organizar a comunicação entre as tarefas, mantendo a reserva dos recursos e tratando eventos inesperados sem iniciar movimentos automaticamente.
Meta de aprendizagem	Ao final, você deverá conseguir acompanhar uma sequência integrada, reconhecer quando cada comando é permitido e explicar como recuperar uma operação bloqueada.

1. Integrar não é apenas juntar os códigos

Copiar vários programas para o mesmo arquivo não garante que eles funcionem em conjunto. É necessário definir quais informações cada tarefa fornece, quem decide e quando cada comando pode ser enviado.

O programa integrado precisa ter apenas um `setup()` e um `loop()`. As funções de sensores, desvios, sinalização e tração são chamadas a partir dessa organização.

Um sensor fornece um evento. A lógica da operação interpreta esse evento conforme a etapa atual. Somente depois dessa decisão são alterados o sinal, o alvo de tração ou a condição de reserva.

2. O percurso didático

Utilizaremos uma composição, em sentido único, com um evento de entrada e outro de saída. O D6 será comandado para AC durante a preparação, antes da circulação.

S10 representa o ponto de entrada e S11 representa um ponto posterior de saída. São nomes didáticos de sensores ligados à gaveta GA3; o exemplo não define seu posicionamento real nem uma rota oficial da EFMN 74.

O comando de tração será apenas apresentado na tela. Na montagem real, a tração analógica está relacionada à gaveta GAR e exige uma interface adequada, como explicado no capítulo 9.

A sequência pressupõe que o sensor de entrada esteja localizado antes da região protegida e que os comandos de redução e parada sejam adequados ao percurso. Nenhuma distância de frenagem é calculada neste exercício.



3. Etapas da operação

Etapa	Condição ou ação
Bloqueada	Tração solicitada em zero; informações precisam de conferência.
Disponível	Informações válidas, região livre e nenhuma reserva.
Preparação	Reservar o recurso e apresentar o comando D6 em AC.
Aguardar posição	Após a espera didática, exigir confirmação manual das agulhas.
Pronta	Desvio conferido; aguarda solicitação de circulação.
Circulação solicitada	Sinal verde e alvo de tração simulado de 30%.
Passagem em andamento	S10 detectado; sinal vermelho e alvo simulado de 15%.
Aguardar liberação	S11 detectado; alvo zero e reserva mantida.

Os percentuais são exemplos de solicitações. Não representam velocidade medida, PWM aplicado aos trilhos ou confirmação de parada física.

4. O que os sensores podem confirmar?

A é o comando que simula uma nova passagem em S10. B simula uma passagem em S11. Cada comando representa um evento reconhecido, e não o nível contínuo da entrada.

Depois de A, a região deixa de ser considerada livre. Depois de B, ela continua sem liberação: o último vagão ainda pode estar no percurso.

O operador precisa conferir a saída de toda a composição e a disponibilidade da região antes de enviar L. Depois, F retira a reserva.

Receber S11 antes de S10, repetir um evento inesperado ou receber uma passagem durante a preparação indica uma sequência inadequada. O programa bloqueia novas etapas, zera o alvo de tração e mantém a reserva existente.

5. Confirmação do desvio e autorização

O comando P prepara o percurso e mostra D6 em AC. Após um segundo de espera didática, o programa aguarda K, que representa uma conferência manual da posição das agulhas.

Essa confirmação manual foi escolhida porque não há sensores Hall nas agulhas dos desvios. O tempo transcorrido e a mensagem de comando não confirmam a posição mecânica.



Depois de K, o operador ainda precisa enviar G para solicitar a circulação. Preparação concluída não inicia a tração automaticamente.

6. Bloqueio e recuperação

E ativa uma emergência simulada. I torna as informações inválidas. Em ambos os casos, o sinal solicitado fica vermelho, o alvo de tração fica em zero e a operação é bloqueada.

A reserva de um percurso preparado permanece. O programa não comanda retorno do D6 para DI e não reutiliza os recursos automaticamente.

C cancela a emergência simulada, mas não recupera a operação. Depois de conferir as informações, a parada física e a disponibilidade da região, o operador utiliza V, L e R.

Se houver reserva, R leva à espera de liberação. F deverá ser enviado para retirar essa reserva antes de uma nova preparação.

7. Comandos da simulação

Comando	Significado
V / I	Informações válidas / invalidar e bloquear.
L	Após conferência, informar região livre.
E / C	Ativar / cancelar emergência simulada.
R	Recuperar após conferir as condições.
P	Reservar e preparar o percurso.
K	Confirmar manualmente a posição do desvio.
G	Solicitar circulação na etapa pronta.
A / B	Simular evento de entrada S10 / saída S11.
F	Retirar a reserva após conferir a liberação.

Utilize somente uma ESP32 de bancada por USB, sem motores, sensores, sinais ou ligação aos trilhos.

8. Programa completo

Crie `intermediate_12_integration`. Copie todos os blocos na ordem para o mesmo programa. Não é necessária biblioteca adicional.



```
enum OperationStage {
    BLOCKED, IDLE, SETTING, WAIT_POSITION, READY,
    RUNNING, PASSING, WAIT_CLEAR
};
OperationStage stage = BLOCKED;
bool informationValid = false;
bool regionClear = false;
bool emergencyActive = false;
bool routeReserved = false;
unsigned long preparationStarted = 0;

bool conditionsKnownAndClear() {
    return informationValid && regionClear && !emergencyActive;
}

void showOutputs(bool green, int targetPercent) {
    Serial.println(green ? "Signal request: GREEN" : "Signal request: RED");
    Serial.print("Traction target: ");
    Serial.print(targetPercent);
    Serial.println("% - simulation only");
}

void blockOperation() {
    stage = BLOCKED;
    showOutputs(false, 0);
    Serial.println("Operation blocked - existing reservation retained");
}

void prepareOperation() {
    if (stage != IDLE || routeReserved || !conditionsKnownAndClear()) {
        Serial.println("Preparation rejected");
        return;
    }
    routeReserved = true;
    stage = SETTING;
    preparationStarted = millis();
    showOutputs(false, 0);
    Serial.println("Simulated command: D6 - AC");
}

void registerPassage(bool entry) {
    if (entry && stage == RUNNING) {
        regionClear = false;
        stage = PASSING;
        showOutputs(false, 15);
        Serial.println("S10 detected - reservation retained");
    } else if (!entry && stage == PASSING) {
        stage = WAIT_CLEAR;
        showOutputs(false, 0);
        Serial.println("S11 detected - whole train clearance NOT confirmed");
    } else {
        Serial.println("Unexpected sensor event");
        regionClear = false;
        blockOperation();
    }
}
```



```
void handleCommand(char command) {
  switch (command) {
    case 'V': informationValid = true; break;
    case 'I': informationValid = false; blockOperation(); break;
    case 'E': emergencyActive = true; blockOperation(); break;
    case 'C': emergencyActive = false; break;
    case 'L':
      if (stage == BLOCKED || stage == IDLE || stage == WAIT_CLEAR) {
        regionClear = true;
        Serial.println("Region declared clear by operator");
      } else Serial.println("Clearance rejected in this stage");
      break;
    case 'R':
      if (stage == BLOCKED && conditionsKnownAndClear()) {
        stage = routeReserved ? WAIT_CLEAR : IDLE;
        Serial.println("Recovered - no automatic movement");
      } else Serial.println("Recovery rejected");
      break;
    case 'P': prepareOperation(); break;
    case 'K':
      if (stage == WAIT_POSITION && conditionsKnownAndClear()) {
        stage = READY;
        Serial.println("Position confirmed by operator - waiting for G");
      } else Serial.println("Position confirmation rejected");
      break;
    case 'G':
      if (stage == READY && routeReserved && conditionsKnownAndClear()) {
        stage = RUNNING;
        showOutputs(true, 30);
      } else Serial.println("Movement request rejected");
      break;
    case 'A': registerPassage(true); break;
    case 'B': registerPassage(false); break;
    case 'F':
      if (stage == WAIT_CLEAR && conditionsKnownAndClear()) {
        routeReserved = false;
        stage = IDLE;
        showOutputs(false, 0);
        Serial.println("Reservation released by operator");
      } else Serial.println("Release rejected");
      break;
    default:
      if (command != '\n' && command != '\r') Serial.println("Invalid command");
      break;
  }
}

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("EFMN 74 - Integrated operation simulation");
  blockOperation();
  Serial.println("Commands: V I L E C R P K G A B F");
}
```



```
void loop() {  
  if (stage == SETTING && millis() - preparationStarted >= 1000) {  
    stage = WAIT_POSITION;  
    Serial.println("Wait ended - check D6 and send K");  
  }  
  if (Serial.available() > 0) handleCommand(Serial.read());  
}
```

9. Como as tarefas se relacionam

registerPassage() recebe os eventos. A variável stage determina se cada evento faz sentido naquele momento. showOutputs() apresenta as solicitações resultantes para sinal e tração.

prepareOperation() mantém a reserva antes de emitir o comando do desvio. O pedido P é recusado durante a circulação, a espera de liberação ou um bloqueio.

O programa não mantém uma fila de pedidos recusados. Retirar uma emergência ou tornar as informações válidas não executa automaticamente P, K ou G.

O alvo zero após S11 é uma solicitação de parada. Antes de L e F, a composição inteira e a parada física precisam ser conferidas. O código não mede a velocidade nem detecta o último vagão.

Exercício prático

Meta: acompanhar o ciclo completo e conferir os bloqueios.

1. Compile, envie e abra o Monitor Serial em 115200 baud. Pressione EN ou RESET.
2. Envie G. Confira a recusa na condição inicial.
3. Simule a conferência da região: envie V, L e R, uma letra de cada vez.
4. Envie P. Confira o comando D6 em AC e a reserva.
5. Aguarde a mensagem de fim da espera. Simule a conferência física das agulhas e envie K.
6. Envie G. Confira sinal verde e alvo de 30%.
7. Envie A. Confira sinal vermelho, alvo de 15% e reserva mantida.
8. Envie P durante a passagem. A nova preparação deve ser recusada.
9. Envie B. Confira alvo zero e ausência de confirmação da saída de toda a composição.
10. Envie F antes de L. A liberação deverá ser recusada.
11. Simule a conferência da saída da composição inteira e envie L e F.
12. Confira a retirada da reserva, com sinal vermelho e alvo zero.

Teste de evento inesperado

Reinicie a simulação, prepare a operação e solicite circulação. Envie B antes de A. O programa deverá bloquear a operação e manter a reserva.

Para recuperar, confira a situação, envie V, L e R e, depois, F. Será necessária nova preparação para voltar a circular.



Teste de emergência

Em outra sequência, envie E durante a circulação. Confira alvo zero, sinal vermelho e reserva mantida. Envie C: nenhum movimento deve ser retomado automaticamente.

Após a conferência de parada e região livre, utilize V, L, R e F. Se as informações não puderem ser confirmadas, mantenha o bloqueio.

10. Integração física e divisão por gavetas

Na montagem real, a leitura dos sensores da gaveta GA3 deverá produzir eventos após a verificação de estabilidade do capítulo 5. O comando do D6 será convertido para seus ângulos calibrados, respeitando as regras do capítulo 11.

O alvo de tração será tratado pela interface da gaveta GAR no modo Analógico ou pela central apropriada no modo DCC. O programa deste capítulo não define a comunicação entre as ESP32 dessas gavetas.

Uma integração distribuída exige definir mensagens, identificação dos comandos, confirmação de recebimento e tratamento de perda de comunicação. Informações antigas não devem ser aceitas como se fossem uma leitura atual.

Organize as funções em módulos: entradas fornecem eventos; a lógica mantém estados e reservas; saídas aplicam apenas os comandos permitidos. Evite que uma função de sensor movimente diretamente um servo ou altere a tração sem essa verificação.

Os percentuais de 30% e 15%, o intervalo de um segundo e os pontos S10/S11 são parâmetros didáticos. A aplicação à maquete exige conferir posições, distâncias, velocidade, composição, alimentação e comportamento em falhas.

Resumo do capítulo

- Integração exige uma sequência definida e responsabilidades claras.
- Preparar o desvio não inicia a circulação.
- A posição das agulhas precisa de conferência adequada.
- Eventos de sensores são interpretados conforme a etapa atual.
- O alvo de tração é uma solicitação, não uma velocidade medida.
- A saída de toda a composição deve ser conferida antes de retirar a reserva.
- Eventos inesperados, emergência ou informações inválidas bloqueiam novas etapas.
- A recuperação não retoma movimentos automaticamente.

Próximo capítulo: Sonorização da ferrovia.